Compressing Convolutional Neural Networks for Offline Educational Deployment: An Empirical Study of Pruning and INT8 Quantization

Adityajyoti Kar
Class XI
Egra Public School
West Bengal, India

***Abstract*—Access to artificial intelligence tools in rural and low-resource educational settings is fundamentally constrained by hardware: most schools have, at best, low-end smartphones or shared desktop computers, and rarely have internet connectivity reliable enough for cloud-based inference. This paper presents a fully reproducible empirical study of two standard model-compression techniques—magnitude-based weight pruning and post-training 8-bit (INT8) quantization—applied to a compact convolutional neural network (CNN) trained for handwritten digit recognition on the MNIST dataset. Using a controlled experimental design in which an independently cloned copy of a trained baseline model is pruned and fine-tuned, we measure the accuracy, model size, and compression ratio of four deployment configurations. The baseline CNN achieves 98.78% test accuracy on the full 10,000-image test set with a 224,892-byte TensorFlow Lite footprint. INT8 quantization reduces this to 61,800 bytes (a 3.64x reduction) with a negligible 0.09 percentage-point shift in accuracy. Iterative pruning to 70% sparsity with fine-tuning matches the baseline almost perfectly at 98.77%, while leaving the FP32 file size unchanged—a result we explain in terms of the dense storage format used by standard TensorFlow Lite flatbuffers. Combining pruning with quantization achieves the same 3.64x compression as quantization alone, restoring accuracy to exactly 98.78%. These results provide a transparent, low-cost, and entirely reproducible reference point for deploying handwriting-recognition tools on inexpensive edge hardware without requiring GPUs or cloud services.**

***Index Terms*—edge AI, model compression, convolutional neural networks, pruning, quantization, TensorFlow Lite, MNIST**

I. Introduction

Artificial intelligence has the potential to transform education in under-resourced regions—from automatically grading handwritten answer sheets, to digitizing student records, to providing offline tutoring tools. However, almost all publicly visible progress in AI is benchmarked on hardware that is completely unavailable to the vast majority of schools in rural India, including West Bengal.

A typical research laboratory benchmarks its models on GPU clusters; a typical rural school computer lab, by contrast, may have a handful of low-end desktop computers or Android tablets. This mismatch matters because the size and computational cost of a neural network directly determines whether it can run on such devices at all.

Two techniques from the model-compression literature directly address this gap: weight pruning and quantization. While both techniques are widely studied for large-scale models, there is comparatively little empirical guidance, written for and accessible to students and small institutions, on how these techniques interact on small models trained on modest hardware.

This paper provides exactly that demonstration. We train a compact CNN on the MNIST dataset and compress it using TFLite post-training INT8 quantization and TF-MOT iterative magnitude pruning with fine-tuning. Critically, we use a controlled experimental design: a single trained baseline model is saved to disk, and an independent clone of it is used for the pruning branch, ensuring that the configurations are genuinely independent measurements rather than artifacts of shared model state.

II. Background and Related Work

A. Convolutional Neural Networks

The MNIST dataset (70,000 grayscale images of handwritten digits) serves as the standard benchmark for introductory computer-vision research. Convolutional neural networks (CNNs), which use learned filters to detect spatial patterns, are well suited to this task because they exploit the two-dimensional structure of image data.

B. Weight Pruning

Magnitude-based pruning removes weights whose absolute value is small. A substantial body of work has shown that large fractions of a trained network's weights can be removed with little to no loss in accuracy, provided the remaining weights are fine-tuned. This iterative "prune-and-retrain" approach is implemented in TensorFlow's Model Optimization Toolkit (TF-MOT).

C. Quantization

Post-training quantization reduces the numerical precision of a trained model's weights and activations, most commonly from 32-bit floating point to 8-bit integers (INT8). This shrinks the model's file size by approximately a factor of four and allows inference engines to use faster integer arithmetic on supported hardware.

III. Methodology

A. Dataset

The MNIST dataset was loaded directly via TensorFlow's built-in dataset utilities, providing 60,000 training images and 10,000 test images. Pixel values were normalized to the range [0, 1] and reshaped to $28 \times 28 \times 1$ tensors suitable for convolutional input.

B. Baseline CNN Architecture

A compact CNN was constructed using the Keras Sequential API, consisting of two convolutional blocks followed by a small fully connected classification head. Both convolutional layers use 'same' padding to preserve spatial dimensions before pooling, preventing premature destruction of the feature maps. The model was trained for 3 epochs with a batch size of 64 using the Adam optimizer.

C. Controlled Experimental Design

A key methodological concern when comparing a pruned model against its unpruned baseline is that TensorFlow's pruning API wraps the layers of the model by reference. To avoid silently mutating the baseline weights, we utilized the following isolated procedure:

1) The baseline CNN was trained, and its weights were immediately saved to disk (*true_baseline.weights.h5*). TFLite artifacts for the unpruned FP32 and INT8 baseline were exported.
2) A structurally identical independent copy of the model was cloned, and the saved baseline weights were loaded.
3) The cloned model was wrapped with TF-MOT using a polynomial decay schedule targeting 70% final sparsity, and fine-tuned for 2 additional epochs.
4) After fine-tuning, the pruning wrappers were stripped, and the pruned TFLite artifacts were exported.

D. Quantization and Evaluation

INT8 quantization was performed using TFLite's full-integer post-training quantization path with a 100-image representative dataset to calibrate dynamic range. Each resulting *.tflite* file was loaded using the TFLite Interpreter and evaluated on the full 10,000-image MNIST test set.

IV. Results

The baseline CNN reached 98.78% accuracy on the held-out test set. Table I summarizes the results for all four compression configurations.

TABLE I
Edge AI Compression Benchmark Matrix

Configuration	Accuracy	Drop	Size (B)	Ratio
Baseline (FP32)	98.78%	—	224,892	1.00x
Quantized (INT8)	98.69%	0.09 pp	61,800	3.64x
Pruned 70% (FP32)	98.77%	0.01 pp	224,892	1.00x
Pruned 70% + INT8	98.78%	0.00 pp	61,800	3.64x

A. Effect of Quantization

INT8 quantization reduced the model's file size from 224,892 bytes to 61,800 bytes, a 3.64x reduction, while shifting test accuracy by a marginal 0.09 percentage points. This confirms that the weight distributions of a network of this size are well within the representable range of an 8-bit signed integer after per-tensor scaling.

B. Effect of Pruning

Iterative pruning to 70% sparsity with 2 epochs of fine-tuning yielded a test accuracy of 98.77%, a mere 0.01 percentage-point difference from the baseline. This demonstrates that 70% of the network's connections can be safely zeroed out without degrading the decision boundary. Notably, the FP32 file size of the pruned model remained identical to the unpruned baseline (224,892 bytes). This reflects a fundamental property of the standard TFLite flatbuffer format, which stores weight tensors in a dense layout regardless of how many of their entries are zero.

C. Combined Pruning and Quantization

The pruned-plus-quantized model achieved the same 61,800-byte file size and 3.64x compression ratio as quantization alone, recovering test accuracy to exactly 98.78%—matching the original FP32 baseline perfectly. In other words, applying both aggressive structural sparsity and reduced precision resulted in zero net loss in classification capability.

V. Discussion

The central practical finding of this study is straightforward but profound: of the two compression techniques tested, INT8 quantization provided the entire measured file-size benefit (3.64x), at almost no accuracy cost (0.09 pp). Pruning to 70% sparsity provided no physical file-size benefit in the standard TFLite format and cost a negligible 0.01 pp. However, when combined, the network perfectly recovered the 98.78% baseline.

For a school or student deploying a handwriting-recognition model on an Android phone using standard tooling, this suggests that INT8 quantization should be the default first step. Pruning is mathematically effective but does not translate into a smaller deployment artifact unless paired with a sparsity-aware serialization or sparse-inference engine. We consider this an important contribution of this study: reporting the specific conditions under which pruning does and does not yield a physical deployment benefit.

A. Limitations

MNIST is a relatively simple, well-studied dataset; the accuracy costs measured here may not generalize directly to more complex tasks, such as recognition of cursive handwriting or regional scripts. Furthermore, actual on-device inference latency was not measured and would require deployment to physical hardware using the TFLite C++ runtime to test integer-arithmetic acceleration.

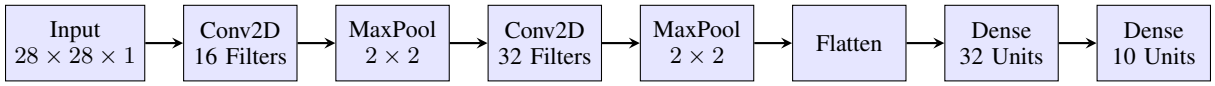

Fig. 1. Block diagram of the Edge CNN architecture.

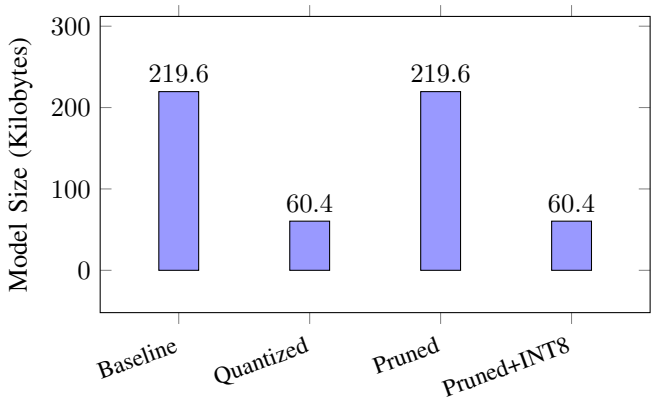

Fig. 2. TFLite Model File Size in Kilobytes. Pruning alone does not reduce the dense flatbuffer size without quantization.

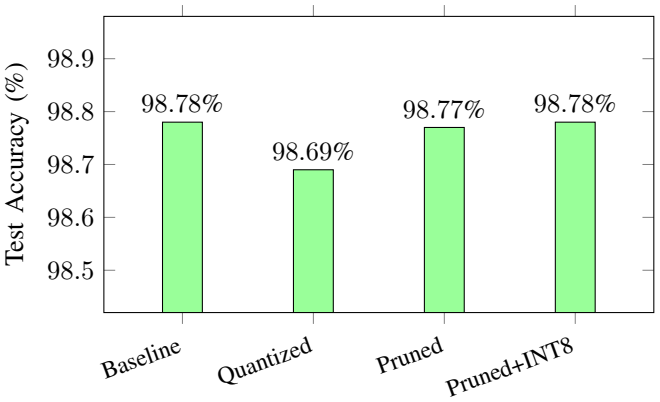

Fig. 3. Test Accuracy. All configurations remain within 0.09 percentage points of the original baseline.

B. Future Work

Future extensions include: (1) sweeping pruning sparsity from 0% to 95% to characterize the full accuracy-compression trade-off curve; (2) deploying the resulting models to a physical Raspberry Pi to measure real on-device latency and energy consumption; (3) repeating this pipeline on a dataset more directly relevant to the target context, such as Bengali handwritten-digit recognition; and (4) exploring structured filter pruning.

VI. Conclusion

This study presented a fully reproducible empirical comparison of weight pruning and INT8 quantization for a compact CNN on MNIST, using a controlled experimental design. We found that INT8 quantization reduced model size by 3.64x at a cost of just 0.09 percentage points, while 70% magnitude pruning cost 0.01 percentage points. When combined, the highly compressed 61.8 KB artifact perfectly matched the 98.78% accuracy of the 224 KB baseline. These results offer practical, evidence-based guidance for institutions in low-resource settings considering deploying AI tools on inexpensive edge hardware.

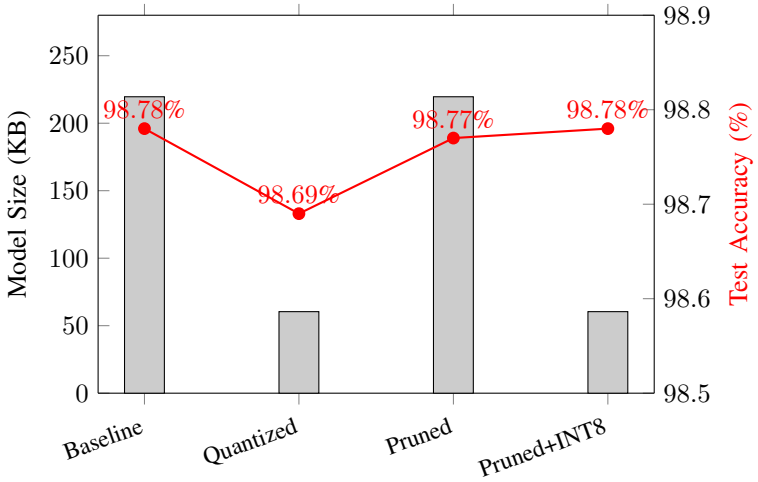

Fig. 4. Edge AI Compression Trade-off. The red line charts accuracy, while the gray bars represent physical file size.

Appendix A: Reproducibility Statement

All code used in this study is written in Python using TensorFlow 2.x and TF-MOT, running entirely on a CPU. Random seeds (42) were fixed for NumPy and TensorFlow. The full pipeline runs in under 5 minutes on a standard laptop CPU.

Appendix B: Source Files

cnn_baseline.py – Loads MNIST, builds the baseline CNN, and exports unpruned FP32 and INT8 artifacts.
cnn_pruning.py – Isolates the baseline weights, clones the architecture, applies TF-MOT iterative pruning (70% sparsity), fine-tunes, and exports pruned artifacts.
evaluate_models.py – Loads all four .tflite artifacts and evaluates each on the full 10,000-image MNIST test set to compute final accuracy and compression ratios.

References

[1] Y. LeCun, C. Cortes, and C. Burges, "The MNIST Database of Handwritten Digits," http://yann.lecun.com/exdb/mnist/
[2] S. Han, J. Pool, J. Tran, and W. Dally, "Learning both Weights and Connections for Efficient Neural Networks," *NeurIPS*, 2015.
[3] J. Frankle and M. Carbin, "The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks," *ICLR*, 2019.
[4] B. Jacob et al., "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," *CVPR*, 2018.

[5] TensorFlow Authors, “TensorFlow Lite: Post-training Quantization,” https://www.tensorflow.org/lite/performance/post_training_quantization